

MSML606 Bonus Project 2 Proposal

Supply Chain Attack Detection via npm Dependency Graph Analysis

Helen Li Yaxita Amin
April 15, 2026

AI Statement: No AI was used for core logic or initial approach. All code, comments, and documentation (README) is or will be authored and typed manually by us.

Team Composition

This project is a group submission by two members:

- Helen Li (UID: 118466200)
- Yaxita Amin (UID: 121292483)

Dataset

Dataset 1: npm Registry API (Dependency Graph Source)

Source: npm public registry, freely accessible at <https://registry.npmjs.org/express>

Scale: A single popular seed package (e.g., `express`, `react`, `lodash`) produces a transitive dependency graph of 500–2,000+ nodes and 1,000–5,000+ edges within 4–6 BFS levels.

Format: JSON responses per package, containing dependency manifests and package-level meta-data.

Key metadata fields include:

- `dependencies` / `devDependencies`: maps of required package names to version ranges
- `dist-tags`: latest published version pointer
- `time`: publish and update timestamps per version
- `maintainers`: current and historical owner information
- Weekly download counts: available via <https://api.npmjs.org/downloads/>

Why this dataset is suitable: The npm registry contains the real-world dependency metadata for over 2 million packages. Crawling it from a seed node produces a large, naturally unstructured directed graph with cycles, isolated subgraphs, and deeply nested transitive dependencies, which is ideal for exercising BFS, DFS, cycle detection, and topological sort at realistic scale.

Dataset 2: OSV (Open Source Vulnerability) Database (Ground-Truth Labels)

Source: Google Open Source Vulnerabilities database: <https://osv.dev>, queryable at <https://api.osv.dev/v1>

Format: JSON vulnerability records keyed by ecosystem and package name.

Key fields include:

- `id`: CVE or GHSA vulnerability identifier

- `affected[].package.name`: the flagged npm package name
- `severity`: CVSS score and rating
- `details`: human-readable description of the vulnerability

Why this dataset is suitable: OSV provides ground-truth labels (known-malicious or known-vulnerable) for npm packages, enabling quantitative evaluation of the flagging system. It directly bridges the gap between raw graph structure and real-world threat intelligence.

Application Idea

Application: npm Transitive Dependency Security Analyzer

The core algorithm is graph traversal applied to the directed dependency graph constructed from the npm registry. The application takes a user-supplied package name, builds its full transitive dependency graph, and flags suspicious or vulnerable sub-dependencies that would otherwise be invisible to the developer.

Core algorithmic components:

- **BFS (Breadth-First Search)**: Used to crawl the npm registry iteratively from a seed package, discovering all transitive dependencies level by level. BFS naturally annotates each node with its *depth* from the root, which directly answers “how many layers deep is this suspicious package hiding?”
- **DFS (Depth-First Search)**: Used to enumerate all root-to-node paths, so the system can report the exact dependency chain leading to any flagged package (e.g., `express` → `body-parser` → `qs` → `[flagged-pkg]`).
- **Cycle Detection**: DFS-based back-edge detection identifies circular dependency chains. The npm ecosystem does not permit publish-time cycles, so any detected cycle is automatically flagged as anomalous.
- **Topological Sort (Kahn’s Algorithm)**: Applied to the DAG after cycle resolution to reveal packages with the highest in-degree — those depended upon by the most other packages in the tree. High-in-degree packages that are also newly published or recently transferred represent the highest-impact attack vector.

What the application does: Given a package name, the system constructs the full dependency graph, traverses it with BFS/DFS, detects cycles, and scores each node using a suspicion heuristic derived from OSV labels, download counts, ownership transfer recency, and typosquatting distance. It reports all flagged packages alongside their exact dependency path and suspicion signals.

Example alert: *“You installed `express`. At depth 4, `event-stream@3.3.6` is flagged: listed in OSV, maintainer ownership transferred 180 days ago, 0 weekly downloads.”*

Why graph algorithms are meaningful here: The dependency tree of a real-world package can contain thousands of nodes that no developer manually audits. Linear inspection is impractical and misses transitive relationships entirely. Graph traversal exposes the full reachable attack surface, cycle detection catches structural anomalies, and topological sort identifies the highest-leverage nodes for targeted auditing.